

3D-визуализация блокчейна Ethereum

React + Three.js (@react-three/fiber) + Ethers.js

Бакланов Егор, БПИ234

Выбранные технологии и мотивация

React: компонентная архитектура и управление состоянием UI.

Three.js + @react-three/fiber: построение интерактивной 3D-сцены прямо в React.

Ethers.js: доступ к Ethereum JSON-RPC и форматирование on-chain данных.

TypeScript: строгая типизация модели графа и RPC-ответов.

Vite: быстрый dev-сервер и удобная production-сборка.

React

React использовался как основной UI-фреймворк для построения интерфейса (панель управления, статусы загрузки, отображение деталей выбранного узла) и управления состоянием приложения.

Выбор пал в его пользу, так как изучался на НИСе (хорошо мне знаком). С Vue не вижу смысла сравнивать, поскольку я им не пользовался, и в любом случае не выбрал бы его.

Three.js + @react-three/fiber

Three.js отвечает за низкоуровневую 3D-графику (геометрии, материалы, источники света, камера, сцена), а `@react-three/fiber` позволяет описывать эту сцену декларативно в стиле React-компонентов.

Этот стек выбран, потому что он сочетает производительность и гибкость Three.js с удобством компонентной архитектуры React.

Альтернативой Three.js был Babylon.js, но гпт сказал, что данный аналог "проигрывает в гибкости интеграции в React-проекты и размере сообщества" (я доверился ему).

Ethers.js

Ethers.js используется для подключения к Ethereum JSON-RPC, получения номера актуального блока, загрузки последних блоков и транзакций, а также для корректного форматирования on-chain значений (Wei -> ETH, Gwei).

Библиотека выбрана из-за лаконичного API и хорошей поддержки TypeScript. В сравнении с Web3.js (опять же со слов гпт), "Ethers.js обычно проще для построения чистого, типобезопасного клиентского кода и имеет более современный DX".

TypeScript и Vite

TypeScript применялся для строгой типизации структур блоков, транзакций и графовых сущностей (узлы/ребра).

По сравнению с чистым JS, TS повышает надежность кода при росте сложности приложения и облегчает поддержку проекта.

Vite использовался как инструмент сборки и dev-сервер.

Выбран по совету учебного ассистента, так как Webpack устарел.

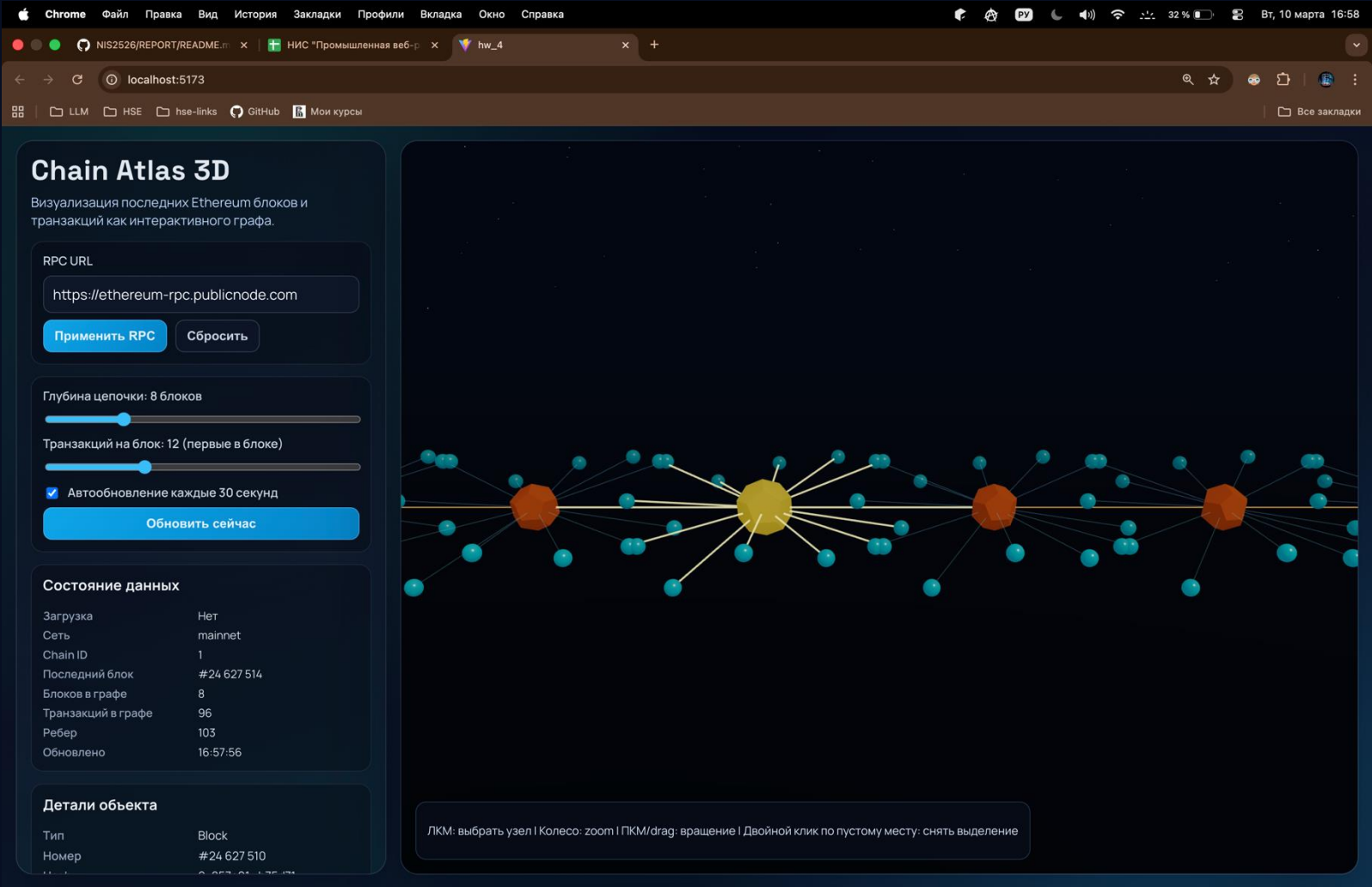
Плюс ко всему данная связка использовалась в предыдущих домашних заданиях.

Суть приложения

Проект представляет собой веб-приложение для визуализации блокчейна Ethereum в виде интерактивного 3D-графа. Каждый блок отображается как отдельный узел основной цепочки, а транзакции внутри блока визуализируются вокруг него как дочерние узлы с ребрами-связями.

Пользователь может вращать сцену, масштабировать камеру, выбирать блоки и транзакции для просмотра деталей (hash, адреса, gas, value, nonce, timestamp), менять глубину цепочки, ограничение по количеству транзакций и RPC endpoint, а также включать автообновление данных.

Скриншот работы приложения



Заключение

Проект показал, что 3D-представление помогает быстрее видеть структуру и плотность активности блоков.

Основной задачей был баланс читаемости графа, производительности и объема отображаемых данных.

Использованный стек позволил реализовать мою задумку: React дал управляемый UI, R3F упростил 3D-слой, Ethers.js закрыл работу с сетью.